Secure Cloud Document Portal

A Multi-Party Encrypted File Storage System

Cryptography Coursework Report

May 17, 2026

Contents

1 Title of Project

The title of this project is **Secure Cloud Document Portal**. It is a multi-party encrypted file storage system. The system uses two symmetric encryption algorithms and one asymmetric encryption algorithm. All algorithms are built from scratch without using any external cryptography libraries. The system also includes a Certificate Authority for identity verification.

2 Introduction and Problem Statement

Cryptography is the science of protecting information. It makes data unreadable to anyone who does not have the correct key. Today we store a lot of sensitive data on cloud servers. This includes medical records and financial documents and personal files. If someone breaks into the server they can read all the data. This is a very serious problem.

There are two main problems this project tries to solve. The first problem is protecting data while it moves from the client to the server. This is called data in transit. An attacker can listen to the network and steal the data if it is not encrypted. The second problem is protecting data while it sits on the server disk. This is called data at rest. If someone gets access to the server files they should not be able to read them.

These problems are very important in the real world. Hospitals need to protect patient records. Banks need to protect financial data. Governments need to protect classified documents. The General Data Protection Regulation also requires companies to protect personal data. A data breach can cost millions of dollars and damage the reputation of a company.

This project builds a complete solution to both problems. It uses Blowfish to encrypt files on the server. It uses ChaCha20 to encrypt data during transmission. It uses ElGamal for key exchange so that no secret keys are sent over the network. A Certificate Authority verifies the identity of the server and clients.

3 Project Objectives

The objectives of this project are listed below.

1. Implement two symmetric encryption algorithms from scratch for secure storage. Blowfish is a block cipher and ChaCha20 is a stream cipher. Both are coded using only basic Python math operations.

2. Implement ElGamal as an asymmetric encryption algorithm for secure key exchange and digital signatures. ElGamal is based on the Discrete Logarithm Problem.

3. Integrate all algorithms into a multi-party communication system. The system has a client and a server and a Certificate Authority. The CA issues and verifies digital certificates.

4. Conduct performance analysis of all implemented algorithms. This includes measuring encryption speed and memory usage and ciphertext size expansion.

5. Provide small number examples that can be verified by hand. These examples help demonstrate understanding of the mathematical foundations.

4 Literature Review

This section reviews ten academic and industry sources related to the cryptographic concepts used in this project.

Schneier [**?**] designed the Blowfish algorithm in 1993. It is a 16 round Feistel network with a 64 bit block size. The key can be between 32 and 448 bits. Schneier showed that Blowfish is fast and secure for software implementations. The key schedule uses the digits of pi to initialize the P array and S boxes.

Bernstein [**?**] designed the ChaCha20 stream cipher as an improvement over Salsa20. ChaCha20 uses Add Rotate and XOR operations instead of S boxes. This makes it very efficient on processors without hardware AES support. Google adopted ChaCha20 for TLS in Chrome and Android [**?**].

ElGamal [**?**] proposed a public key cryptosystem based on the Diffie Hellman key exchange. The security depends on the difficulty of the Discrete Logarithm Problem. ElGamal provides both encryption and digital signatures. It is widely used in the GNU Privacy Guard software.

Diffie and Hellman [**?**] introduced the concept of public key cryptography. Their key

exchange protocol allows two parties to create a shared secret over an insecure channel. This idea is the foundation of modern secure communication.

Stallings [?] provides a comprehensive textbook on cryptography and network security. He explains symmetric and asymmetric algorithms and their applications. He also covers key management and certificate authorities in detail.

Menezes et al. [?] wrote the Handbook of Applied Cryptography. This book gives detailed mathematical descriptions of all major cryptographic algorithms. It includes implementation guidelines and security analysis.

Barker [?] published NIST Special Publication 800-57 on key management recommendations. This document defines minimum key lengths and key lifecycle management practices. It recommends at least 128 bits for symmetric keys and 2048 bits for asymmetric keys.

Cooper et al. [?] defined the X.509 certificate format in RFC 5280. This standard describes how Certificate Authorities issue and verify digital certificates. Our project simulates this process using ElGamal signatures.

Bernstein and Lange [?] reviewed post quantum cryptography approaches. They discuss how quantum computers could break current public key systems like ElGamal. Lattice based algorithms are promising replacements.

Avanzi et al. [?] proposed CRYSTALS Kyber as a post quantum key encapsulation mechanism. NIST selected Kyber as a standard in 2022. This shows the importance of preparing for quantum threats.

The gap this project addresses is the lack of educational implementations that combine multiple algorithms into a working system. Most textbooks explain algorithms in isolation. This project shows how they work together in a real application.

5 Methodology

5.1 Implementation Approach

All encryption algorithms are implemented from scratch in Python. No external cryptography libraries are used. Only the Python standard library is used. This includes `hashlib`

for SHA-256 hashing and `os.urandom` for random number generation and `sqlite3` for database storage.

Blowfish is implemented as a 16 round Feistel cipher. The P array has 18 entries and there are four S boxes with 256 entries each. The constants come from the hexadecimal digits of pi. The cipher supports ECB mode and CBC mode with PKCS7 padding.

ChaCha20 is implemented using a 4 by 4 matrix of 32 bit words. The quarter round function performs Add Rotate and XOR operations. Ten double rounds give 20 rounds total. The keystream is XORed with the plaintext to encrypt.

ElGamal is implemented with safe prime generation using the Miller Rabin primality test. The modular exponentiation uses the square and multiply algorithm. The extended Euclidean algorithm computes modular inverses.

5.2 Verification with Small Numbers

The project includes a small number verification script. It uses ElGamal with $p = 23$ and $g = 5$ and $x = 6$. The public key is $y = g^x \bmod p = 5^6 \bmod 23 = 8$. For encryption with message $m = 7$ and random $k = 3$ the ciphertext is computed as follows.

$$c_1 = g^k \bmod p = 5^3 \bmod 23 = 10 \tag{1}$$

$$c_2 = m \cdot y^k \bmod p = 7 \cdot 8^3 \bmod 23 = 19 \tag{2}$$

For decryption the shared secret is $s = c_1^x \bmod p = 10^6 \bmod 23 = 2$. The modular inverse is $s^{-1} \bmod 23 = 12$. The recovered message is $m = c_2 \cdot s^{-1} \bmod p = 19 \cdot 12 \bmod 23 = 228 \bmod 23 = 7$. This matches the original message. These small numbers can be checked with a calculator or by hand.

For digital signatures the verification equation is $g^h \equiv y^r \cdot r^s \pmod p$. The project includes a script that computes each step and shows the values. This makes it easy to verify the signature by hand.

The Blowfish verification shows a simplified 2 round Feistel network. Each round performs XOR with a subkey and then applies the F function and then swaps the halves. The ChaCha20 verification shows one quarter round with the four ARX steps.

5.3 Performance Evaluation

Performance is measured using three metrics. Encryption speed is measured in milliseconds for data sizes of 1 KB and 10 KB and 100 KB. Memory usage is measured using the `tracemalloc` module. Ciphertext expansion compares the size of plaintext to ciphertext.

An end to end benchmark measures the time for each phase. This includes CA setup and server start and client handshake and file upload and file download and key rotation. All benchmarks use the `time` module for timing.

5.4 Testing

The project has six test modules with tests for every component. Unit tests verify that encryption followed by decryption returns the original data. Tests check that different keys produce different ciphertexts. Tests check that different IVs produce different ciphertexts. An integration test verifies the full flow from CA setup through file upload and download to key rotation.

6 Use Case and Scenario Design

6.1 Business Scenario

The scenario is a secure document portal for a company. The company has employees who need to upload and download confidential files. A Certificate Authority verifies the identity of the server and each client. The server stores all files in encrypted form.

There are three parties in the system.

- **Certificate Authority.** Generates a root ElGamal keypair. Issues signed certificates to the server and clients. Verifies certificates and manages revocation.
- **Server.** Generates its own ElGamal keypair. Gets a certificate from the CA. Manages user accounts in a SQLite database. Stores files encrypted with Blowfish CBC. Handles key rotation.
- **Client.** Generates its own ElGamal keypair. Gets a certificate from the CA. Connects to the server and performs a handshake. Uploads and downloads files over the

encrypted channel.

6.2 Data Flow

The communication flow has the following steps.

1. The CA generates its root keypair and is ready to issue certificates.
2. The server generates its keypair and receives a certificate from the CA.
3. The client generates its keypair and receives a certificate from the CA.
4. The client connects to the server via TCP socket.
5. The server sends its public key and certificate to the client.
6. The client verifies the server certificate with the CA.
7. The client sends its public key to the server.
8. Both sides compute the shared secret using Diffie Hellman. The shared secret is $g^{x_1 \cdot x_2} \bmod p$.
9. Both sides derive a ChaCha20 session key from the shared secret using SHA-256.
10. All further messages are encrypted with ChaCha20.
11. The user registers and logs in. Passwords are stored as salted SHA-256 hashes.
12. File upload. The file travels over the ChaCha20 channel. The server then encrypts it with Blowfish CBC and stores it on disk.
13. File download. The server decrypts the file from Blowfish. Then it sends the file over the ChaCha20 channel.

6.3 Key Generation and Distribution

Keys are generated and distributed as follows.

ElGamal keys. Each party generates a safe prime $p = 2q + 1$ where q is also prime. A generator g is found for the group. A random private key x is chosen. The public key is $y = g^x \bmod p$. These keys are used for the handshake only.

Session key. The shared secret from the handshake is hashed with SHA-256 to produce a 32 byte ChaCha20 key. A 12 byte nonce is derived similarly. This key is used for one session only.

Storage key. A random 128 bit key is generated for Blowfish. It is stored in a file on the server. Key rotation generates a new key and re-encrypts all stored files.

7 Ethical and Legal and Professional Considerations

No real sensitive data is used in this project. All test data is generated randomly or uses dummy text. No personal information is collected or stored.

The project follows the principles of the General Data Protection Regulation. Data is encrypted both in transit and at rest. Access requires authentication. Passwords are never stored in plaintext.

The key sizes used in this project are small for demonstration purposes. The ElGamal keys are 256 bits for speed during demos. In a production system the keys should be at least 2048 bits as recommended by NIST.

This project is for educational purposes only. It should not be used to protect real sensitive data in production. Professional cryptographic libraries have been tested and audited by experts. This implementation has not been audited.

Responsible cryptographic practices include using random IVs for every encryption and using salted password hashing and supporting key rotation. The system does not store or log any plaintext passwords.

8 Conclusion

This project demonstrates a complete secure cloud document portal. It implements Blowfish and ChaCha20 as symmetric algorithms and ElGamal as an asymmetric algorithm. All algorithms are coded from scratch without any external cryptography libraries.

The system solves two important problems. Data in transit is protected by ChaCha20 encryption. Data at rest is protected by Blowfish CBC encryption. The ElGamal key exchange establishes session keys without sending secrets over the network. The Certificate

Authority provides identity verification through digital signatures.

The project is feasible and fully functional. The automated demo runs all ten phases successfully. The test suite passes all tests. The benchmarks show that ChaCha20 is faster than Blowfish for large data. Blowfish has padding overhead because it is a block cipher. ChaCha20 has zero expansion because it is a stream cipher.

The expected security benefit is dual layer encryption. Even if an attacker breaks one layer the data is still protected by the other layer. The key rotation feature allows the server to change the storage key without losing access to files.

This project provides a strong educational foundation for understanding applied cryptography. The small number examples allow hand verification of every algorithm. The modular design makes it easy to study each component independently.

References

[1] B. Schneier. Description of a New Variable Length Key 64 Bit Block Cipher called Blowfish. *Fast Software Encryption*. Springer. 1994.

[2] D. J. Bernstein. ChaCha a variant of Salsa20. *Workshop Record of SASC*. 2008.

[3] A. Langley and W. Chang and N. Mavrogiannopoulos and J. Strombergson and S. Josefsson. ChaCha20 Poly1305 Cipher Suites for Transport Layer Security. *RFC 7905*. 2016.

[4] T. ElGamal. A Public Key Cryptosystem and a Signature Scheme Based on Discrete Logarithms. *IEEE Transactions on Information Theory*. Vol 31. No 4. 1985.

[5] W. Diffie and M. Hellman. New Directions in Cryptography. *IEEE Transactions on Information Theory*. Vol 22. No 6. 1976.

[6] W. Stallings. *Cryptography and Network Security Principles and Practice*. 7th Edition. Pearson. 2017.

[7] A. Menezes and P. van Oorschot and S. Vanstone. *Handbook of Applied Cryptography*. CRC Press. 1996.

[8] E. Barker. Recommendation for Key Management Part 1 General. *NIST Special Publication 800-57*. 2020.

[9] D. Cooper and S. Santesson and S. Farrell and S. Boeyen and R. Housley and W. Polk. Internet X.509 Public Key Infrastructure Certificate and CRL Profile. *RFC 5280*. 2008.

[10] D. J. Bernstein and T. Lange. Post Quantum Cryptography. *Nature*. Vol 549. 2017.

[11] R. Avanzi et al. CRYSTALS Kyber Algorithm Specifications and Supporting Documentation. *NIST Post Quantum Cryptography Standardization*. 2019.